

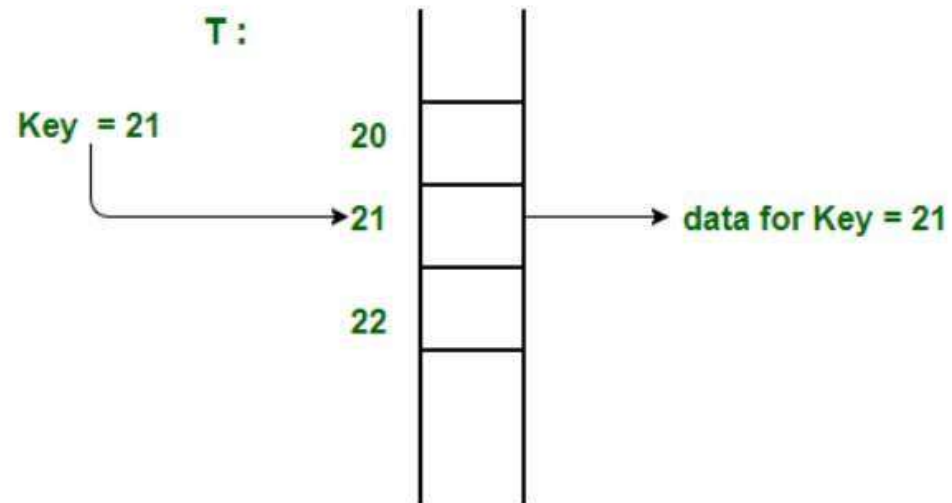
23CS201 - Data Structures and Algorithms using C++

Study Materials

Direct Address Table:

- Direct Address Table is a data structure that has the capability of mapping records to their corresponding keys using arrays.
- In direct address tables, records are placed using their key values directly as indexes. They facilitate fast searching, insertion and deletion operations.
- We can understand the concept using the following example. We create an array of size equal to maximum value plus one (assuming 0 based index) and then use values as indexes.
- For example, in the following diagram key 21 is used directly as index.

Direct Address Table:



Data Address Table:

Data Address Table in Hashing



Direct Address Table:

- **Data Storage:** Direct Address Tables allocate a fixed-size array, where each slot or index in the array corresponds directly to a unique key.
- This establishes a direct mapping between keys and their associated data elements.
- **Data Retrieval:** When you need to retrieve a specific data element, you simply use the key as an index to access the corresponding slot in the array.
- This direct access ensures rapid retrieval, making Direct Address Tables highly efficient for applications that require frequent data lookups.
- **Data Insertion and Deletion:** To insert a new data element, you place it in the slot corresponding to its key. For deletion, you clear the slot associated with the key, effectively removing the element.

Limitations:

- Prior knowledge of maximum key value
- Practically useful only if the maximum value is very less.
- It causes wastage of memory space if there is a significant difference between total records and maximum value.

Advantages:

- **Searching in $O(1)$ Time:** Direct address tables use arrays which are random access data structure, so, the key values (which are also the index of the array) can be easily used to search the records in $O(1)$ time.
- **Insertion in $O(1)$ Time:** We can easily insert an element in an array in $O(1)$ time. The same thing follows in a direct address table also.
- **Deletion in $O(1)$ Time:** Deletion of an element takes $O(1)$ time in an array. Similarly, to delete an element in a direct address table we need $O(1)$ time.

Structure of Direct Address Table:

- **Array:** The core of a Direct Address Table is an array (or list) of fixed size. The size of the array depends on the range of possible key values.
- **Key-to-Index Mapping:** In a Direct Address Table, the keys themselves are used as indices to access the associated data elements.
- Each unique key maps directly to a specific index in the array.
- **Data Storage:** Data elements or values are stored directly in the array at the index corresponding to their associated keys.
- If there is no associated data for a particular key, the corresponding slot in the array is left empty (often represented as None or null).

Hash Function:

- A hash function is a function that takes an input (or 'message') and returns a fixed-size string of bytes.
- The output, typically a number, is called the hash code or hash value.
- The main purpose of a hash function is to efficiently map data of arbitrary size to fixed-size values, which are often used as indexes in hash tables.

Key Properties of Hash Functions:

- **Deterministic:** A hash function must consistently produce the same output for the same input.
- **Fixed Output Size:** The output of a hash function should have a fixed size, regardless of the size of the input.
- **Efficiency:** The hash function should be able to process input quickly.
- **Uniformity:** The hash function should distribute the hash values uniformly across the output space to avoid clustering.
- **Pre-image Resistance:** It should be computationally infeasible to reverse the hash function, i.e., to find the original input given a hash value.
- **Collision Resistance:** It should be difficult to find two different inputs that produce the same hash value.
- **Avalanche Effect:** A small change in the input should produce a significantly different hash value.

Types of Hash Functions:

- There are many hash functions that use numeric or alphanumeric keys.
- Division Method.
- Multiplication Method
- Mid-Square Method
- Folding Method
- Cryptographic Hash Functions
- Universal Hashing
- Perfect Hashing

Applications of Hash Functions:

- Hash Tables: The most common use of hash functions in DSA is in hash tables, which provide an efficient way to store and retrieve data.
- Data Integrity: Hash functions are used to ensure the integrity of data by generating checksums.
- Cryptography: In cryptographic applications, hash functions are used to create secure hash algorithms like SHA-256.
- Data Structures: Hash functions are utilized in various data structures such as Bloom filters and hash sets.